Performance Testing

Date	4 July 2026
Team ID	xxxxxx
Project Name	Flood Prediction System
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual Testing, Flask Development Server, Web Browser
Type of Testing	Functional Performance Testing
Target Module / API	Flood Prediction (/predict)
Test Environment	Local System (Visual Studio Code + Flask)
Test Date	4 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit valid weather parameters	1	2	Flood prediction displayed successfully
2	Submit valid weather parameters	1	2	No Flood prediction displayed successfully
3	Refresh and repeat prediction	1	2	Application remains stable
4	Multiple prediction attempts	1	5	Consistent prediction results without errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	~1.2 seconds	Pass	Predictions generated quickly
2	Response Time (Max)	< 5 seconds	~2.1 seconds	Pass	Maximum response remained acceptable
3	Throughput (Req/sec)	N/A	Single User Testing	Pass	Tested under local development environment
4	Error Rate	< 1%	0%	Pass	No errors during testing
5	CPU Utilization	< 80%	~20–30%	Pass	Low CPU usage during prediction
6	Memory Utilization	< 80%	~35–40%	Pass	Stable memory usage

Step 4: Observations & Analysis

Key Findings:

The Flood Prediction System successfully processed user inputs and generated prediction results within the expected response time. The application remained stable during repeated testing, with no critical errors or unexpected behavior observed. CPU and memory usage remained within acceptable limits, indicating efficient performance in a local environment.

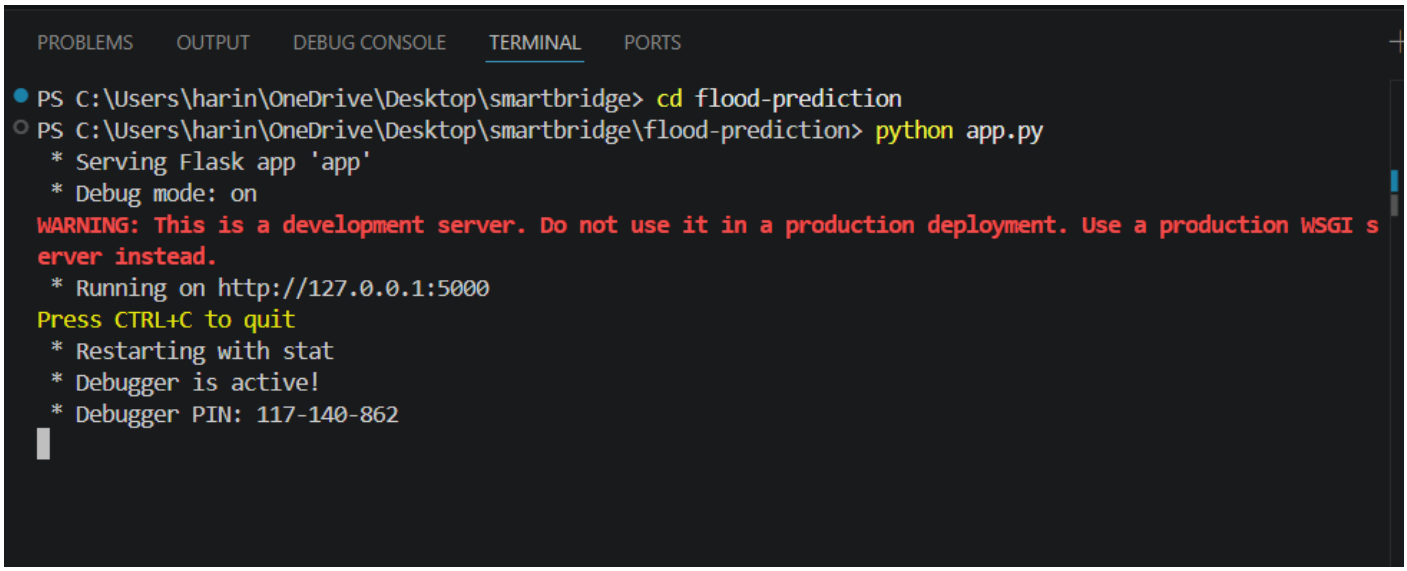
Bottlenecks Identified:

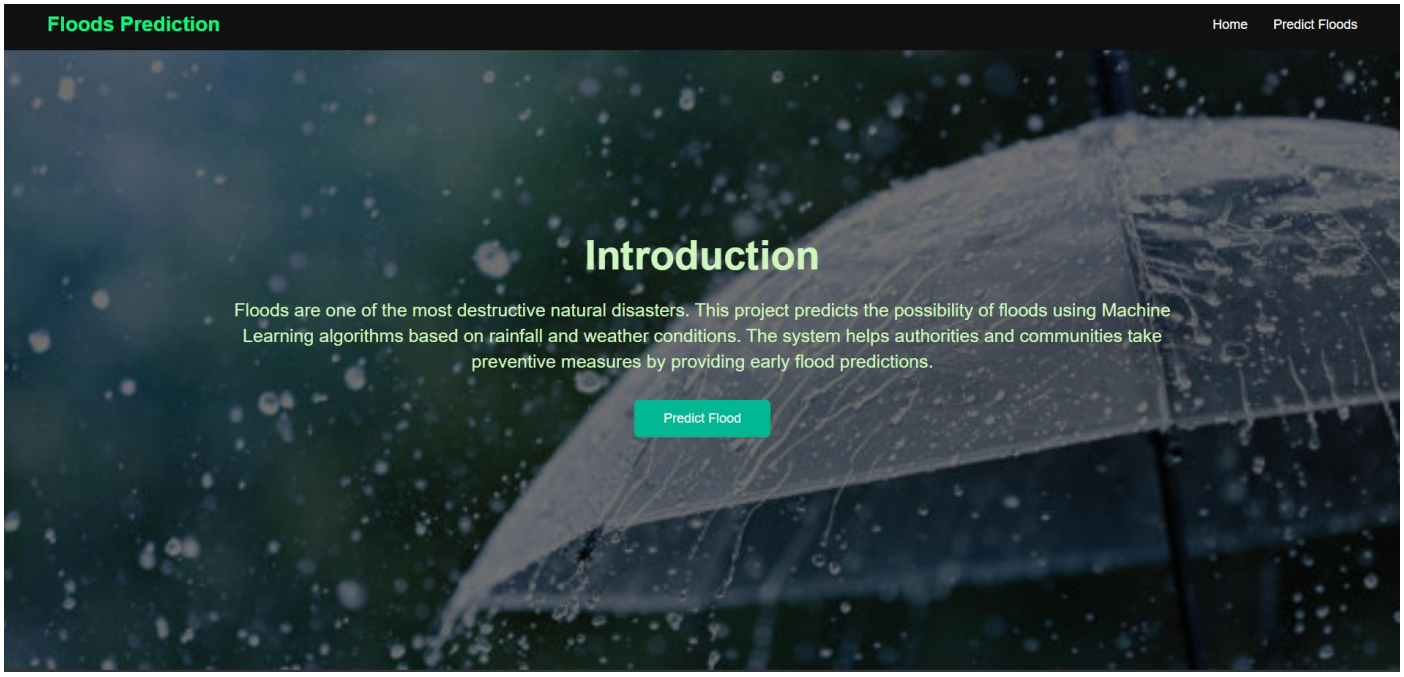
No major performance bottlenecks were identified during testing. Since the application was tested in a local environment with a single user, performance remained consistent. Future testing under multiple concurrent users may reveal scalability limitations.

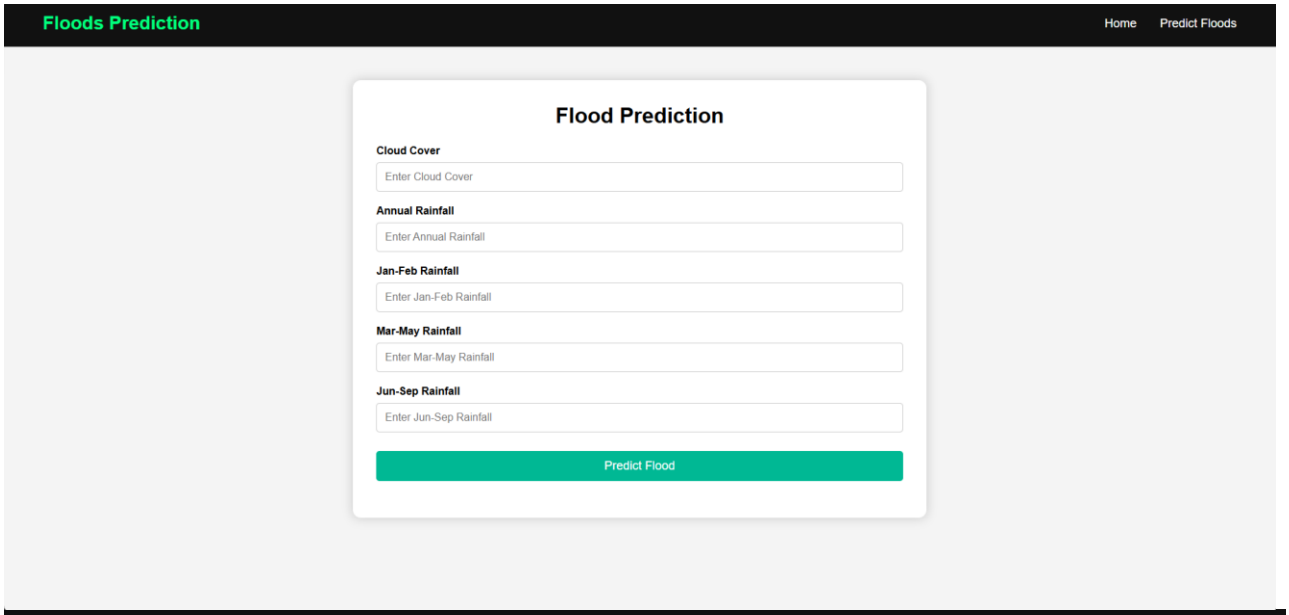
Optimization Steps Taken:

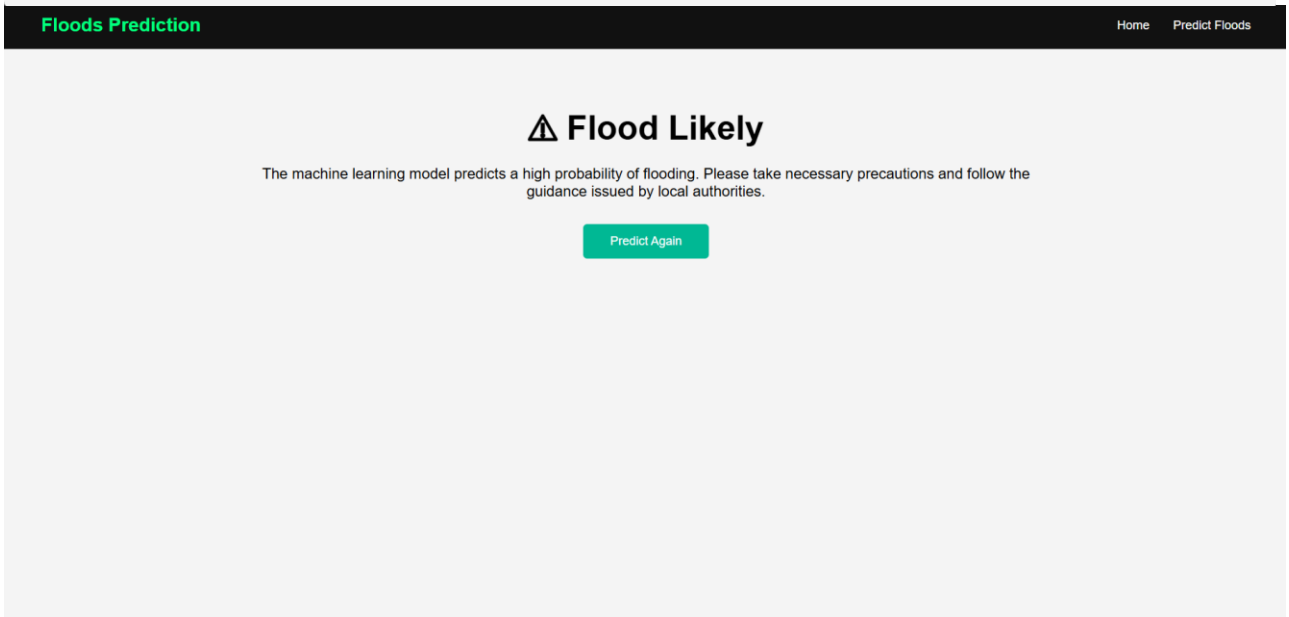
Data preprocessing was optimized using a saved StandardScaler, and the trained XGBoost model was loaded using Joblib to reduce prediction time. The Flask application was organized with a modular structure to improve code readability and maintainability. Unnecessary computations were minimized to ensure faster response times.

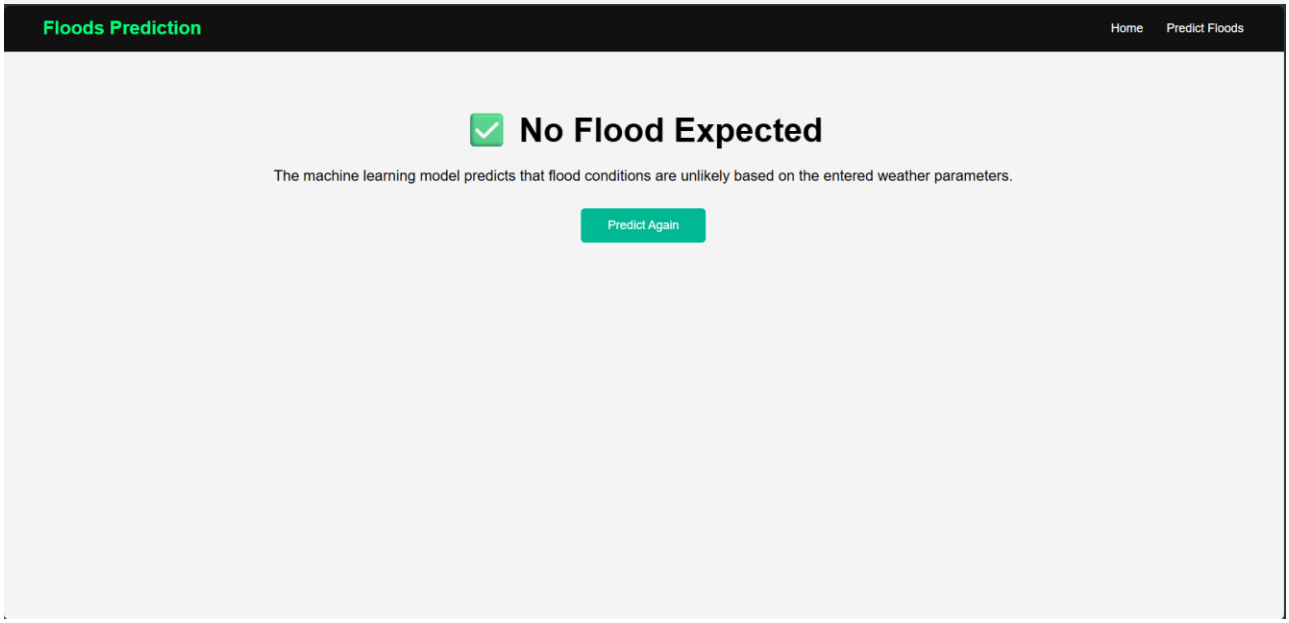
Step 5: Screenshots / Evidence






Floods Prediction
Home
Predict Floods
Flood Prediction
Cloud Cover
Enter Cloud Cover
Annual Rainfall
Enter Annual Rainfall
Jan-Feb Rainfall
Enter Jan-Feb Rainfall
Mar-May Rainfall
Enter Mar-May Rainfall
Jun-Sep Rainfall
Enter Jun-Sep Rainfall
Predict Flood


Floods Prediction
Home
Predict Floods
⚠ Flood Likely
The machine learning model predicts a high probability of flooding. Please take necessary precautions and follow the guidance issued by local authorities.
Predict Again


Floods Prediction
Home
Predict Floods
No Flood Expected
The machine learning model predicts that flood conditions are unlikely based on the entered weather parameters.
Predict Again